Architecture Specification — "Local-First" Multi-Agent System

Version 2.1 (Revised post-audit, second review pass) — Paperclip · Hermes · Goose

> **Status:** Revised version incorporating every finding from the *Architecture Audit Report: Viability of the "Local-First" Multi-Agent Infrastructure*, plus corrections from the **v2.0 second review pass** (Goose transport command/auth/binding, multi-profile port registry, watchdog liveness semantics, `.gitignore` secrets coverage). **Major changelog vs v1:** mandatory HTTP/ACP gateway IPC (end of direct native script spawning), RCE security addressed from the MVP onward, symmetric `GOOSE_PATH_ROOT` isolation, mitigations for Paperclip Issues #1160 / #3819 / #6596, SQLite `journal_mode: delete` strategy, PostgreSQL lock watchdog. **Changelog v2.0 → v2.1:** Goose HTTP server is `goose serve` (not `goose acp`, which is stdio-only) with `GOOSE_SERVER__SECRET_KEY` auth, default port 3284, `/acp` endpoint, and mandatory non-localhost bind; new **ADR-06** (profile→port registry); Phase 1 E2E corrected (no sandbox before Phase 3); watchdog marked-run-failed semantics; `.gitignore` now covers `storage/goose_data/**`. **Feasibility score: v1 = 4/10 → v2 target = 8/10** (conditional on completing Phases 1–3).

Table of Contents

1. Vision and Objectives
2. System Architecture
3. Settled Architecture Decisions (ADRs)
4. Phased Implementation Plan
5. Implementation References (code)
6. Decision Matrices
7. Risk Register
8. Go/No-Go Validation Checklist

---

1. Vision and Objectives

The fundamental objective remains: reconcile the robustness of centralized orchestration with the flexibility of local execution — **without excessive software isolation, but also without security holes**. The system must be:

- **Local-first**: no mandatory external network dependency (apart from cloud LLM APIs in Phase 1);

- **Portable**: Linux, macOS, Windows/WSL2 — a single Git repository plus one local `.env` file;
- **Secure by default**: all AI-generated code is sandboxed, every destructive action is traceable;
- **Resilient**: no silent failure modes ("silent failures") anywhere in the control loop.

1.1 The two v1 pitfalls (and their resolution)

v1 Pitfall	v2 Resolution
Docker ↔ Host topological rupture: a container cannot `spawn` a host process via `run.sh`/`run.ps1`.	**Explicit network IPC gateway**: Hermes in API server mode (`hermes gateway run`, port 8642), Goose in HTTP/ACP server mode (`goose serve`), with Paperclip using the `hermes_gateway` adapter pointing at `host.docker.internal`.
Illusion of globalized isolation: `HERMES_HOME` alone does not cover Goose (Rust, XDG paths).	**Mandatory symmetric injection** of `HERMES_HOME` **and** `GOOSE_PATH_ROOT` per profile, via bootstrap wrappers.

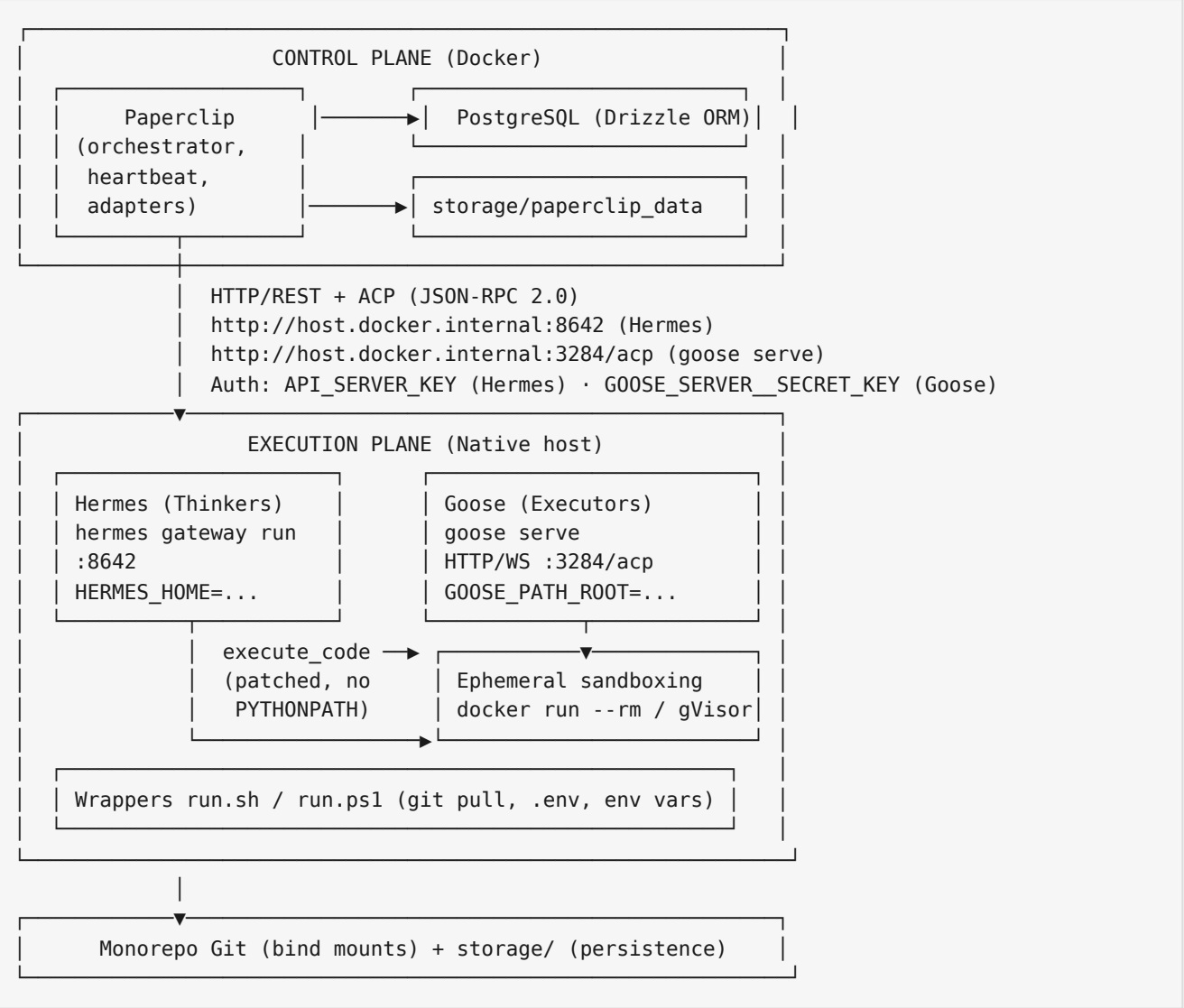
1.2 Separation of the two planes

- **Control Plane (Docker)**: Paperclip — heartbeat-based scheduling, governance, budgets, transactional state (PostgreSQL + Drizzle ORM). Generates no LLM tokens itself.
- **Execution Plane (Native host)**: Hermes (strategy, reasoning) and Goose (technical execution) — direct access to local runtimes (Node.js, Rust, Go, Python), Git, IDEs — **with** confinement of risky code execution.

---

2. System Architecture

2.1 Overview



2.2 Control Plane — Paperclip (Docker)

- **Role**: CEO/HR of the virtual organization. Models agents as employees: ticket assignment, status auditing, dependency management, deliverable validation.
- **Scheduling**: heartbeat model — agents are woken by a scheduler or on events (task assignment, comment mention); each heartbeat carries a *context packet* (company state, project goals, ticket history).
- **Atomicity**: optimistic locking on `checkoutRunId` (409 Conflict if already claimed) — **complemented by an orphaned-lock watchdog** (see §3.5).
- **Adapters**: exclusively `hermes_gateway` (HTTP/SSE to a remote Hermes) plus a REST/ACP client for Goose. **The `hermes_local` adapter is forbidden inside the container** (it can only spawn within its own namespace).
- **Persistence**: bind mounts to `storage/paperclip_data`; PostgreSQL as an external service or dedicated Docker volume.

2.3 Execution Plane — Hermes (Thinkers)

- **Role**: problem decomposition, functional structuring, action planning, critical review of produced work.
- **Server mode**: `hermes gateway run` — resident HTTP API on port **8642**, authenticated via `API_SERVER_KEY`.
- **Identity and memory**: frozen document-injection model at session start — `SOUL.md` (personality), `AGENTS.md`/`.hermes.md` (project conventions), `USER.md` (operator profile), `MEMORY.md` (autonomously learned knowledge). Any manual edit to these files requires a session restart.
- **State**: fully localized under `HERMES_HOME` (YAML config, memories, `state.db` SQLite with FTS5).
- ⚠️ **`execute_code`**: tool is **patched** (removal of `PYTHONPATH` from the inherited environment) and, longer-term, redirected to a containerized sandbox. See §3.2.

2.4 Execution Plane — Goose (Executors)

- **Role**: technical workforce — system commands, AST manipulation, refactoring, build scripts, test suites.
- **Server mode**: **`goose serve`** — the HTTP/WebSocket ACP server (Agent Client Protocol, JSON-RPC 2.0) exposing `initialize`, `session/load`, `request_permission` at the **`/acp`** endpoint.
- ⚠️ **`goose acp` is a *different* command** — it serves ACP over **stdio** (for editor clients such as Zed) and **cannot** accept network connections. The Control Plane must target `goose serve` only. (Reference: aaif-goose/goose issue #6642, "goosed to ACP-over-HTTP".)
- **Defaults**: listens on `127.0.0.1:3284`. Two mandatory overrides for this architecture: `GOOSE_SERVER__HOST=0.0.0.0` (a localhost bind silently refuses traffic from `host.docker.internal`) and `GOOSE_SERVER__PORT` per profile (ADR-06).
- **Auth**: clients authenticate with **`GOOSE_SERVER__SECRET_KEY`** — a distinct secret from Hermes's `API_SERVER_KEY`; both are stored in the local `.env`.
- **Extensibility**: MCP (Model Context Protocol) — tool discovery without recompilation. Crates: `goose-acp`, `goose-cli`, `goose-mcp`.
- **Isolation**: Goose **completely ignores** `HERMES_HOME`. Path resolution uses the `etcetera` crate (XDG standards, `~/.config/goose/`). **Per-profile isolation strictly requires `GOOSE_PATH_ROOT`.**
- **Permissions**: `GOOSE_MODE` ∈ {`auto`, `smart_approve`, `approve`, `chat`}, refined by `tool_permissions.json` (regex-based silent approval/blocking, e.g. blocking paths containing `.env`/`secrets/`). Native sub-agents via the `delegate` tool.

2.5 Memory and Isolation (HERMES_HOME and GOOSE_PATH_ROOT)

Cognitive-context isolation is achieved through **symmetric environment-variable routing** — never through separate machines:

1. Every launched agent inherits **two** variables: - `HERMES_HOME=<repo>/storage/hermes_data/agent_<profile>` - `GOOSE_PATH_ROOT=<repo>/storage/goose_data/agent_<profile>`

2. **Injecting `GOOSE_PATH_ROOT` is mandatory** for every Goose profile. Without it, concurrent Goose instances read/write the same `~/.config/goose/config.yaml` and the same `sessions.db` SQLite database → guaranteed state collision.
3. Each profile directory encapsulates: local working memory (session files, learning history), allowed tools and "skills" for that specific profile, contextual LLM configurations.
4. **SQLite**: any database residing on a Docker volume mount (virtiofs/9p — Docker Desktop on macOS/Windows) must be configured with **`journal_mode: delete`**. The default WAL mode on such mounts causes stale inode retention by the server process and the fatal `DeletedWalGenerationError` (Issue #109740), which blocks all new session creation.

> **Golden rule:** *profile isolation ≠ execution security*. Environment variables compartmentalize identities and memories; they provide **zero protection** against arbitrary code execution (see §3.2).

2.6 Portability and Synchronization

- **Git monorepo**: agent hierarchy, prompt templates, bootstrap scripts, orchestration rules — all versioned.
- **Local `.env`** (excluded from version control): absolute file paths, API keys, `API_SERVER_KEY`, `GOOSE_SERVER__SECRET_KEY`.
- **Reproducibility**: `git clone` + `.env` instantiation + `./run.sh bootstrap` = operational state.

2.7 Responsibility Table

Layer	Component	Runtime	Primary Responsibility
Control Plane	Paperclip	Docker	Governance, task allocation, heartbeat, budgets, status tracking
Execution Plane	Hermes	Native (HTTP server :8642)	Strategy, analytical decomposition, architecture design, critical review
Execution Plane	Goose	Native (`goose serve`, HTTP/WS `:3284/acp`)	Code writing, native shell commands **(sandboxed)**, tests, refactoring
Persistence	Monorepo + bind mounts	Native / Docker	State synchronization, skill versioning
Security	Ephemeral sandbox	Docker/Podman	Confinement of all AI-generated code execution

---

3. Settled Architecture Decisions (ADRs)

These decisions are **settled** (status: *Accepted*) based on the audit. They are no longer "open questions".

ADR-01 — IPC: HTTP / ACP API Gateway (rejection of direct native execution)

- **Decision**: all Control Plane ↔ Execution Plane communication goes through a network channel: Hermes `hermes gateway run` (:8642) + Goose `goose serve` (HTTP/WS, `:3284/acp`, JSON-RPC 2.0), targets `host.docker.internal`, authenticated with `API_SERVER_KEY` (Hermes) and `GOOSE_SERVER__SECRET_KEY` (Goose).
- **Transport caution (v2.1)**: `goose acp` (stdio) ≠ `goose serve` (HTTP/WS). Using the stdio binary behind a systemd unit gives a daemon that listens on nothing — the integration test in §8 exists precisely to catch this class of error.
- **Rationale**: a container can only spawn processes within its own namespace; bind mounts share files but provide no cross-execution vector. Network-based coupling is the only viable architecture for reliable, modular, ACP/MCP-compliant distributed orchestration, with native streaming (SSE) support.
- **Rejected alternatives**:
- *Docker-out-of-Docker* (mounted socket): potential privilege escalation (host root), asymmetric path/permission complexity, unresolved file-locking issues → **rejected**.
- *Local SSH connector*: strong cryptographic isolation but unacceptable per-heartbeat latency and heavy key management → **fallback option only**.
- **Operational consequence**: host daemons must be kept alive via process managers (**systemd** / **launchd** / **pm2**) — see §5.3.

ADR-06 — Multi-profile port registry (single URL per adapter is not enough)

- **Decision**: one resident server per **profile** requires one **port per profile**. A single `HERMES_GATEWAY_URL` / `GOOSE_ACP_URL` cannot address `agent_ceo` and `agent_dev` simultaneously. A central **`ports.env`** (git-tracked template + local override) declares the mapping:
- `HERMES_PORT_CEo=8642`, `HERMES_PORT_<profile>=8643…` → consumed by the systemd/launchd units *and* by Paperclip's `adapters.json` (each adapter instance targets `http://host.docker.internal:<profile port>`).
- Same scheme for Goose (`GOOSE_PORT_DEV=3284`, …).
- `run.sh`/`run.ps1` read the same file and pass the port to the engine (Goose: `GOOSE_SERVER__PORT`; Hermes: gateway port flag/config).
- **Rationale**: without this registry, the Phase 1 "two concurrent Goose profiles" collision test cannot even be constructed over HTTP, and systemd template units would launch multiple profiles all binding the same port (second instance crash-loops).
- **Schema caution (v2.1)**: the `HERMES_GATEWAY_URL` / `GOOSE_ACP_URL` environment-variable names in §5.2 are **illustrative** — verify against Paperclip's actual adapter configuration schema (the documented location is `control-plane/paperclip/config/adapters.json`); the URLs and secrets are the durable part of the contract, the variable names are not.

ADR-02 — Security: human approval in MVP, ephemeral sandboxing as target, immediate RCE patch

Three measures, **all mandatory and complementary**:

1. **Emergency patch (immediate, before any deployment)** — Hermes `execute_code`: - Explicitly remove `PYTHONPATH` from environment variables passed to the subprocess (CVE-2026-9368: module injection via `PYTHONPATH` → loading of `hermes_cli.config` → Base64-encoded exfiltration of API keys, bypassing stdout redaction filters). - Systematically sanitize the inherited environment (`env -i` equivalent: allowlist of variables).
2. **MVP phase (transitional)** — `GOOSE_MODE=smart_approve` (per-tool risk evaluation) or `approve` (systematic confirmation): human control over every destructive action. Accepted cost: breaking asynchronous autonomy (productivity bottleneck), tolerated while trust in the models is being calibrated.
3. **Target solution** — **ephemeral container sandboxing** (`docker run --rm`, Podman, or gVisor) for all unverified code execution (Goose `developer` extension, Hermes `execute_code`): full host isolation, automatic cleanup, restricted network. Once the robust sandbox is deployed, return to `GOOSE_MODE=auto` **coupled with strict token budgets managed by Paperclip**.

> Environmental profile isolation (ADR-03) is retained **for memory only**; it offers no protection against RCE.

ADR-03 — Isolation: symmetric injection of HERMES_HOME + GOOSE_PATH_ROOT

- **Decision**: bootstrap wrappers (`run.sh` / `run.ps1`) and the Paperclip service **always and symmetrically** inject `HERMES_HOME` and `GOOSE_PATH_ROOT` per profile (e.g. `storage/hermes_data/agent_ceo`, `storage/goose_data/agent_dev`). No agent starts without both variables set (hard failure otherwise).
- **Rationale**: Goose is impermeable to `HERMES_HOME`; `HERMES_HOME` alone lets two Goose profiles overwrite each other's `config.yaml`, `secrets.yaml`, permissions, and `sessions.db`.

ADR-04 — SQLite: `journal_mode: delete` on Docker mounts

- **Decision**: any Hermes/Goose SQLite database residing on a Docker volume mount (virtiofs/9p — Docker Desktop macOS/Windows) is forced to `journal_mode: delete`. On native Linux filesystems, WAL may be retained.
- **Rationale**: WAL on virtual mounts → `DeletedWalGenerationError` (Issue #109740), ghost locks, agent memory neutralized.

ADR-05 — Paperclip mitigations: persistSession disabled + lock watchdog

Issue	Symptom	Decision
#1160 — Session ID corruption (`hermes_local`, overly permissive regex `^\d{8}_\d{6}_[a-zA-Z0-9]+$` → invalid ID persisted → `--resume` crash "Session not found" → infinite loop of 2-second runs)	Infinite loop, blocked task	**`persistSession: false`** in the Hermes adapter configuration **until** an upstream fix strictly validates the ID format. Reactivation planned in Phase 4.
#3819 — Orphaned locks (`checkoutRunId` not cleared after silent failure → "Silent Agent	Progress blocked with	**Watchdog**: automatic purge of `checkoutRunId` for any run inactive for

Issue	Symptom	Decision
Stall")	no visible error	**> 60 minutes** (scheduled SQL routine or priority API endpoint). See §5.4.
#6596 — Offline request storm (massive run creation during prolonged disconnection)	Adapter log saturation	**Exponential backoff** on retries + suspension of analytical reviews upon detection of low-level network errors (`ENOTFOUND`, `ECONNREFUSED`).

---

4. Phased Implementation Plan

> Governing principle from the audit: **communication (IPC) and material security foundations must be consolidated BEFORE any advanced cognitive feature is deployed.**

Phase 1 — Topological consolidation, IPC, and stabilization (Month 1) — MVP

Objective: complete Control Plane ↔ Execution Plane pipeline, minimally secured, on cloud APIs (Anthropic/OpenAI). Out of scope: local inference, Qdrant.

Deliverables:

1. **Monorepo** (tree in §5.1) with strict `.gitignore`.
2. **Dockerized Control Plane**: `docker-compose.yml` with external PostgreSQL/dedicated volume, Drizzle ORM configured.
3. **Persistent host daemons**: `systemd`/`launchd` services keeping `hermes gateway run` (:8642) and `goose serve` (HTTP/WS, per-profile port, **`GOOSE_SERVER__HOST=0.0.0.0`**) running in the background (ADR-01, ADR-06).
4. **`hermes_gateway` adapter**: URL `http://host.docker.internal:8642`, `API_SERVER_KEY` set. HTTP/ACP client for Goose at `http://host.docker.internal:3284/acp` with `GOOSE_SERVER__SECRET_KEY` (ADR-06 port registry).
5. **Mitigation #1160**: `persistSession: false`.
6. **Watchdog #3819**: purge of orphaned `checkoutRunId` locks (> 60 min).
7. **Bootstrap wrappers** `run.sh` / `run.ps1` with **symmetric** `HERMES_HOME` + `GOOSE_PATH_ROOT` injection (ADR-03).
8. **MVP security**: `GOOSE_MODE=smart_approve`, `PYTHONPATH` patch on `execute_code` (ADR-02.1 and 2.2).
9. **SQLite**: `journal_mode: delete` on any Docker mount (ADR-04).
10. **Network backoff** (Issue #6596).

E2E validation (phase exit criteria):

1. `docker compose up -d` → Paperclip healthy.
2. Create a task via Paperclip UI/API: "*Generate a Python math utility module and its unit tests*".

3. CEO agent (`run.sh ceo`) → technical specification ticket written to `execution-plane/workspace`.
4. Dev agent (`run.sh dev`) → reads the spec, writes the Python script, runs `pytest` **on the host under `GOOSE_MODE=smart_approve` human confirmation** (ephemeral sandboxing arrives in Phase 3 — do not claim sandboxed execution before then).
5. Confirmation reported back to Paperclip; no corrupted sessions; no residual orphaned locks.

Phase 2 — State isolation and profile compartmentalization (Months 1–2)

1. **Instantiation wrapper** dynamically generating compartmentalized data trees and injecting `HERMES_HOME`/`GOOSE_PATH_ROOT` per role (`/agents/profiles/researcher` vs `/builder`).
2. **SQLite concurrency audit** across all virtual-mount deployments; enforce `journal_mode: delete`.
3. **Versioned standardized `tool_permissions.json`**: blocking regexes (`*.env*`, `secrets/**`, `rm -rf`, etc.), human approval required for destructive instructions.
4. Collision tests: two concurrent Goose profiles on the same workspace → verified zero state sharing.

Phase 3 — Hardening and sandboxing (Month 2)

1. **Complete `execute_code` patch**: environment allowlist, `PYTHONPATH` removal, execution audit logging (definitive closure of the CVE-2026-9368 attack surface).
2. **Ephemeral sandboxing**: compilation/test operations redirected to a disposable containerized MCP server (restricted network, `--rm`, workspace mounted with project-only read/write). Protection against `rm -rf`, system key tampering, exfiltration.
3. **Controlled return to `GOOSE_MODE=auto`** + strict Paperclip-managed token budgets.

Phase 4 — Advanced modularity, RAG, and subagents (Month 3+)

1. **Hierarchical delegation**: Goose's `delegate` tool — asynchronous sub-agents for code analysis / documentation generation, significantly accelerating overall project processing.
2. **Re-enable `persistSession: true`** after upstream Paperclip fix validating the session-ID regex.
3. **RAG**: dedicated MCP server + vector database (Qdrant in the Control Plane), replacing static `MEMORY.md`/`USER.md` memorization — essential for codebases exceeding the models' context window.
4. **Obsidian Knowledge Vault**: shared Markdown knowledge base (company directives, operational syntheses).
5. **Skills Discovery**: Hermes queries Qdrant to dynamically load the most relevant prompts/tools for the current task (lightening the initial context window).

Phase 5 — Local inference and scalability

- **Native Ollama** on the host (CUDA / Apple Metal); local models (Llama 3, DeepSeek-Coder, Qwen) for routine tasks; cloud fallback or reserved for highly strategic tasks.
- Drastic API cost reduction; full offline autonomy.

---

5. Implementation References (code)

5.1 Monorepo tree

```
local-first-agents/
├── .env.example
├── .gitignore
├── docker-compose.yml
├── control-plane/
│   └── paperclip/
│       └── config/
│           ├── adapters.json        # hermes_gateway, persistSession:false
│           └── tool_permissions.json
├── execution-plane/
│   ├── scripts/
│   │   ├── run.sh                   # POSIX bootstrap
│   │   ├── run.ps1                  # Windows bootstrap
│   │   ├── ports.env.example        # profile → port registry (ADR-06)
│   │   ├── watchdog-locks.sh        # checkoutRunId purge (#3819)
│   │   └── systemd/                 # hermes-gateway@.service, goose-serve@.service
│   ├── profiles/
│   │   ├── ceo/   (hermes_config/)
│   │   └── dev/   (goose_config/)
│   └── workspace/
└── storage/
    ├── paperclip_data/
    ├── hermes_data/
    │   ├── agent_ceo/
    │   └── agent_dev/
    └── goose_data/
        ├── agent_dev/
        └── agent_reviewer/
```

Mandatory `.gitignore`:

> ⚠️ v2.1: `storage/goose_data/**` was **missing** — it contains `secrets.yaml` (plaintext) and `sessions.db`. Committing agent state is a credential-leak path (R8).

```
.env
ports.env
storage/hermes_data/**/sessions/
storage/hermes_data/**/logs/
storage/goose_data/**          # secrets.yaml (plaintext), sessions.db — NEVER commit
storage/paperclip_data/
execution-plane/workspace/*
!execution-plane/workspace/.gitkeep
```

5.2 `docker-compose.yml` (Control Plane)

```
version: "3.8"
services:
  paperclip:
    image: paperclip/orchestrator:latest
    container_name: paperclip_control_plane
    restart: unless-stopped
    ports:
      - "3000:3000"
    environment:
      - NODE_ENV=production
      - STORAGE_PATH=/app/storage
      # Execution Plane targets via the gateway — NEVER local spawning
      # NOTE: env var names are illustrative — the authoritative config surface is
      # control-plane/paperclip/config/adapters.json (verify against Paperclip's schema).
      - HERMES_GATEWAY_URL=http://host.docker.internal:8642
      - GOOSE_ACP_URL=http://host.docker.internal:3284/acp   # goose serve (HTTP/WS ACP)
      - API_SERVER_KEY=${API_SERVER_KEY}
      - GOOSE_SERVER__SECRET_KEY=${GOOSE_SERVER__SECRET_KEY}
    extra_hosts:
      - "host.docker.internal:host-gateway"   # required on Linux
    volumes:
      - ./storage/paperclip_data:/app/storage
      - ./control-plane/paperclip/config:/app/config
      - ./execution-plane/workspace:/app/workspace
```

5.3 Host systemd service — Goose ACP example

```
# /etc/systemd/system/goose-serve@.service
# One instance per profile: systemctl start goose-serve@dev
[Unit]
Description=Goose ACP-over-HTTP server (profile %i)
After=network.target

[Service]
Type=simple
# NOTE: replace with the actual service user — do not hardcode personal usernames
User=%i
WorkingDirectory=/home/%i/local-first-agents
EnvironmentFile=/home/%i/local-first-agents/execution-plane/scripts/ports.env
Environment=GOOSE_PATH_ROOT=/home/%i/local-first-agents/storage/goose_data/agent_%i
Environment=GOOSE_MODE=smart_approve
# HTTP/WS ACP server — NOT `goose acp` (stdio-only, for editor clients)
Environment=GOOSE_SERVER__HOST=0.0.0.0
# Per-profile port + secret are injected from the ADR-06 registry, e.g. via a
# per-instance drop-in: systemctl edit goose-serve@dev  →
#   [Service] Environment=GOOSE_SERVER__PORT=3284
#             Environment=GOOSE_SERVER__SECRET_KEY=<from .env / protected file>
# (In production the secret MUST live in a protected EnvironmentFile, never inline.)
ExecStart=/usr/local/bin/goose serve
Restart=on-failure
RestartSec=5

[Install]
WantedBy=default.target
```

⚠️ **Binding warning**: `goose serve` defaults to `127.0.0.1`, which silently refuses all traffic from the Paperclip container (`host.docker.internal`). `GOOSE_SERVER__HOST=0.0.0.0` + `GOOSE_SERVER__SECRET_KEY` + host firewall rules are **all mandatory** together. (Port and secret per profile come from the ADR-06 `ports.env` registry — the `%i` placeholders above are illustrative.)

(Hermes equivalent: `hermes gateway run` with `HERMES_HOME=.../storage/hermes_data/agent_%i`, profile-specific port (ADR-06), `API_SERVER_KEY` loaded from a protected `EnvironmentFile=`.)

5.4 Orphaned-lock watchdog (Issue #3819)

⚠️ **Schema caution**: table/column names below are **illustrative** — validate against the actual Drizzle schema before deploying (see `doc/SPEC-implementation.md`).

⚠️ **Liveness semantics (v2.1)**: a 60-minute-old lock is *not* automatically orphaned — legitimately long runs exist. The purge must therefore (a) check run liveness (no heartbeat/activity update, or process no longer running), and (b) mark the stale run **`failed`** rather than silently reopening the ticket as `ready` — otherwise two agents can end up working the same ticket (duplicate-execution risk).

```
-- Run by cron/systemd timer every 10 minutes
-- (Illustrative; adjust identifiers to the real Drizzle schema.)
UPDATE runs
SET status = 'failed', finished_at = now(),
    error = 'watchdog: orphaned lock — no activity for 60+ minutes'
WHERE finished_at IS NULL
  AND started_at < now() - interval '60 minutes'
  AND last_activity_at < now() - interval '60 minutes';  -- liveness check, not just start time

UPDATE tickets
SET checkout_run_id = NULL, status = 'ready'
WHERE checkout_run_id IS NOT NULL
  AND checkout_run_id IN (
    SELECT id FROM runs WHERE status = 'failed' AND error LIKE 'watchdog:%'
  );
```

5.5 `execute_code` patch (CVE-2026-9368)

```
import os

_ENV_ALLOWLIST = {"PATH", "HOME", "LANG", "TMPDIR", "VIRTUAL_ENV"}

def _sanitized_env() -> dict:
    """Strict allowlist: neutralizes PYTHONPATH and any module-injection
    vector (CVE-2026-9368)."""
    env = {k: v for k, v in os.environ.items() if k in _ENV_ALLOWLIST}
    env.pop("PYTHONPATH", None)  # redundant, defense in depth
    env["PYTHONPATH"] = ""       # explicitly empty
    env["PYTHONDONTWRITEBYTECODE"] = "1"
    return env

# subprocess.Popen(..., env=_sanitized_env(), ...)
```

5.6 `run.sh` (POSIX bootstrap, symmetric injection)

```
#!/usr/bin/env bash
set -euo pipefail
PROFILE="${1:?usage: run.sh <profile> [hermes|goose]}"
ENGINE="${2:-auto}"
ROOT="$(cd "$(dirname "$0")/../.." && pwd)"

git -C "$ROOT" pull --rebase
set -a; source "$ROOT/.env"; set +a

export HERMES_HOME="$ROOT/storage/hermes_data/agent_$PROFILE"
export GOOSE_PATH_ROOT="$ROOT/storage/goose_data/agent_$PROFILE"
mkdir -p "$HERMES_HOME" "$GOOSE_PATH_ROOT"

case "$ENGINE" in
  hermes) exec hermes gateway run ;;
  goose)
    # HTTP/WS ACP server — NOT `goose acp` (stdio-only, editor clients)
    export GOOSE_SERVER__HOST="${GOOSE_SERVER__HOST:-0.0.0.0}"
    export GOOSE_SERVER__PORT="${GOOSE_SERVER__PORT:?port required (see ADR-06 ports.env)}"
    exec goose serve ;;
  auto)   [[ -d "$ROOT/execution-plane/profiles/$PROFILE/goose_config" ]] \
            && exec "$0" "$PROFILE" goose || exec "$0" "$PROFILE" hermes ;;
  *) echo "unknown engine: $ENGINE" >&2; exit 1 ;;
esac
```

(`run.ps1`: identical logic — `git pull --rebase`, `.env` loading, `$env:HERMES_HOME` + `$env:GOOSE_PATH_ROOT`, launching `hermes gateway run` / `goose serve` with `GOOSE_SERVER__HOST`/`GOOSE_SERVER__PORT` set.)

5.7 `tool_permissions.json` (standardized excerpt)

⚠️ **v2.1 caution**: verify this file's schema against the installed Goose version before relying on it — Goose's *native* permission mechanism is `config.yaml` (`GOOSE_MODE`, per-tool permission settings). If `tool_permissions.json` is not supported by the deployed build, express the same rules there (or via a small pre-exec wrapper); the *policy content* below is the durable part.

```
{
  "default_mode": "smart_approve",
  "rules": [
    { "pattern": ".*(\\.env|secrets/|id_rsa|\\.ssh/).*", "action": "block" },
    { "pattern": "^rm\\s+-rf\\s+/", "action": "block" },
    { "pattern": "^git push(--force)?\\s+.*main$", "action": "require_approval" },
    { "pattern": "^pytest|^npm test|^cargo test", "action": "allow" }
  ]
}
```

---

6. Decision Matrices

Matrix 1 — IPC Protocols (decision: A)

Alternative	Deployment	Advantages	Drawbacks	Verdict
A. HTTP/REST/ACP API Gateway ✅	`hermes_gateway` (:8642) + `goose serve` (HTTP/WS, `:3284/acp`, `GOOSE_SERVER__SECRET_KEY`)	Maximum loose coupling; natively crosses the Docker barrier; SSE streaming; ACP/MCP compliant	Requires systemd/pm2 to keep daemons alive; `goose serve` must bind `0.0.0.0` (not its localhost default)	**ADOPTED**
B. Unix sockets / Docker-out-of-Docker	Docker socket mounted into the control container	Very low latency; full subprocess lifecycle control	Critical security risk (potential root escalation); asymmetric path/permission complexity	**REJECTED**
C. Local SSH connector	SSH loopback per heartbeat	Strong cryptographic isolation; granular auth	Unacceptable per-heartbeat latency; heavy key management	**FALLBACK ONLY**

Matrix 2 — Security / Sandboxing (decisions: patch + B + C)

Strategy	Mechanism	Advantages	Drawbacks	Verdict
Env patch (`PYTHONPATH`)	Environment allowlist in `execute_code`	Immediately neutralizes CVE-2026-9368	Doesn't cover other vectors	**MANDATORY (immediate)**
A. Environmental profiles	`HERMES_HOME` + `GOOSE_PATH_ROOT`	Zero CPU/RAM cost; cognitive isolation	No hardware/kernel barrier; useless against RCE	**YES — memory only**
B. Ephemeral containers	`docker run --rm` / disposable MCP server / gVisor	Total host isolation; auto cleanup	Workspace sync complexity; startup latency	**YES — absolute requirement (target)**
C. Human approval	`GOOSE_MODE=smart_approve`/`approve`	Total discretionary control over destructive flows	Breaks asynchronous autonomy (bottleneck)	**YES — transitional (MVP)**

7. Risk Register

#	Risk	Severity	Mitigation	Status
R1	RCE via `PYTHONPATH` (CVE-2026-9368) → API key exfiltration	**Critical**	Env patch + ephemeral sandbox + key rotation	Phase 1 (patch) / Phase 3 (sandbox)
R2	Infinite session-ID loop (#1160)	High	`persistSession: false`	Phase 1
R3	Orphaned PostgreSQL locks (#3819) — silent stall	High	60-min watchdog + manual purge endpoint	Phase 1
R4	Multi-Goose state collision (`~/.config/goose/`)	High	Mandatory symmetric `GOOSE_PATH_ROOT` (hard fail if absent)	Phase 1–2
R5	`DeletedWalGenerationError` on virtual mounts	Medium	`journal_mode: delete`	Phase 1
R6	Offline run storm (#6596)	Medium	Exponential backoff + `ENOTFOUND`/`ECONNREFUSED` detection	Phase 1
R7	`GOOSE_MODE=auto` without sandbox → destructive commands	**Critical**	`smart_approve` in MVP; sandbox before returning to `auto`	Phase 1 → 3
R8	`.env` leak (plaintext keys) into the repo	Critical	`.gitignore` + `GOOSE_SERVER__SECRET_KEY`/`API_SERVER_KEY` in protected env files + rotation	Ongoing
R9	`goose serve` unreachable from container (default `127.0.0.1` bind / wrong command `goose acp` used)	**High**	`GOOSE_SERVER__HOST=0.0.0.0` + per-profile port (ADR-06) + Go/No-Go curl test from *inside* the container	Phase 1
R10	Watchdog purges a legitimately long run → duplicate execution	Medium	Liveness check (`last_activity_at`) + mark run `failed`, not `ready`	Phase 1

8. Go/No-Go Validation Checklist

Phase 1 (MVP) can only be declared complete if:

- [ ] `host.docker.internal:8642` responds from inside the Paperclip container (curl + `API_SERVER_KEY`).
- [ ] `goose serve`'s `/acp` endpoint answers the `initialize` handshake (JSON-RPC 2.0) from **inside** the container — confirms `GOOSE_SERVER__HOST=0.0.0.0` and correct port; a timeout here means a localhost bind or stdio command in use.
- [ ] Two Goose profiles launched on **distinct ports** (ADR-06) → zero cross-writes to configs (collision test).
- [ ] Two Goose profiles launched concurrently → zero cross-writes to configs (collision test).
- [ ] `PRAGMA journal_mode` returns `delete` for every database on a Docker mount.

- [ ] `execute_code` runs with an empty `PYTHONPATH` (test: `import hermes_cli` must fail from the sandbox).
- [ ] `persistSession: false` verified in the adapter configuration.
- [ ] The watchdog purges a test lock older than 60 minutes.
- [ ] Full E2E CEO → Dev → `pytest` → Paperclip confirmation, with no ghost runs.
- [ ] `tool_permissions.json` blocks access to `.env` and `secrets/` (negative test).

Appendix A — Audit Finding → v2 Mapping

Audit Finding	v2 Section	Status
Docker ↔ Host IPC gap	ADR-01, §2.1, §5.2–5.3	Fixed
CVE-2026-9368 / RCE	ADR-02, §5.5, Phase 3	Fixed
Goose ignores `HERMES_HOME`	ADR-03, §2.5, §5.6	Fixed
`DeletedWalGenerationError` (WAL/virtiofs)	ADR-04	Fixed
Issue #1160 (session ID)	ADR-05, Phase 1.5 / Phase 4.2	Mitigated, then resolved
Issue #3819 (orphaned locks)	ADR-05, §5.4	Mitigated
Issue #6596 (offline run storm)	ADR-05, Phase 1.10	Mitigated
Unacceptable deferred sandboxing	ADR-02.3, Phase 3	Fixed
v2.1 review — `goose acp` is stdio; HTTP server is `goose serve` (`:3284/acp`, `GOOSE_SERVER__SECRET_KEY`, `0.0.0.0` bind required)	§2.4, ADR-01, §5.2–5.3, R9	Fixed
v2.1 review — no per-profile port mapping	ADR-06, §5.3, §8	Fixed
v2.1 review — watchdog risked killing long runs / reopening duplicates	§5.4, R10	Fixed
v2.1 review — `storage/goose_data/**` missing from `.gitignore` (secrets leak)	§5.1, R8	Fixed
v2.1 review — Phase 1 E2E claimed sandboxed pytest (sandbox is Phase 3)	Phase 1 E2E	Fixed
v2.1 review — unverified env-var names / schemas flagged for validation	ADR-06, §5.4, §5.7	Flagged

Appendix B — Primary Sources (from the audit)

1. Paperclip — Adapters overview (paperclipai/paperclip, docs/adapters/overview.md)
2. Goose — Configuration Files (goose-docs.ai)
3. Goose — Sandbox Analysis Report (agent-safehouse.dev)
4. Hermes — execute_code sandbox exposure (NousResearch/hermes-agent #7071)
5. CVE-2026-9368 — hermes-agent RCE (SentinelOne / GHSA-wm96-9gfh-vvgq)
6. Paperclip #3819 — `checkoutRunId` not cleared

7. hermes_local adapter setup (youcanbuildthings.com) 25–26. Hermes Configuration / #109740 — `DeletedWalGenerationError` 31–33. Agent Client Protocol, goose ACP, `sacp` crate
8. Goose Permission Modes (goose-docs.ai)
9. Paperclip — HERMES_GATEWAY_ONBOARDING.md
10. Paperclip #1160 — invalid session ID
11. Paperclip #6596 — heartbeat recovery / connectivity

Added in v2.1 (second review pass):

47. aaif-goose/goose #6642 — "project: goosed to ACP-over-HTTP" (confirms `goose acp` = stdio, `goose serve` = HTTP/WS ACP server)
48. Goose docs — Running a Remote goose Server / GDK ACP (`goose serve` listens on `127.0.0.1:3284`, endpoint `/acp`, auth via `GOOSE_SERVER__SECRET_KEY`)